AutoIA Software Manual

User Guide, Operations Reference, and Developer Maintenance Guide

Version 0.21.0.dev1

August 2026

Abstract

This manual explains how to install, use, diagnose, test, and modify AutoIA version 0.21.0.dev1. It is written for two overlapping audiences. The first is an engineering user who wants to run validated interval analysis on ordinary Python/NumPy code. The second is a researcher/developer who needs to understand the source layout well enough to add a function, representation, semantic primitive, controller policy, performance backend, or many-box execution feature without violating the certification architecture.

All code examples are complete Python fragments intended to be directly copyable. The manual states the supported-language boundary explicitly; code that happens to work outside that boundary should not be treated as a stable Version-1 contract.

Contents

1 Installation and project layout **4**

2 First validated calculation **4**

3 Specifying uncertain inputs **5**

4 Choosing a representation **5**

5 Polynomial order **6**

6 Supported scalar arithmetic **6**

7 Supported elementary and statistical functions **6**

8 SciPy compatibility **7**

9 NumPy arrays and ordinary engineering assembly **8**

10 Broadcasting, reductions, and matrix composition **8**

11 Norms **9**

12 Semantic linear solves **9**

13 Uncertain Python branches **10**

14 Branch policy and resource limits **10**

15 Range policies **10**

16 Prescribed subdivision **11**

17 Adaptive subdivision **11**

18 Understanding result objects **12**

19 Understanding pathwise results **12**

20 Prepared sessions for repeated queries **12**

21 Certificate cache behavior **13**

22 Many-box finite collections **14**

23 CPU parallel many-box execution **14**

24 Streaming millions of boxes **15**

25 Session statistics **15**

26 Trusted reduction backend configuration **16**

27 Typed errors and how to respond **16**

28 Current supported-language boundaries **17**

29 Known boundary issue for sqrt/log **17**

30 Performance guidance for users **18**

31 Developer setup and regression commands **18**

32 Source tree for developers **18**

33 Adding a new supported elementary function **19**

34 Adding a new semantic primitive **20**

35 Adding or changing a representation **20**

36 Changing trusted arithmetic **21**

37 Changing the many-box cache or parallel backend **21**

38 GPU development guidance **21**

39 Release checklist for a code change **22**

40 Complete copyable example: nonlinear solve, branch, statistics, and repeated queries **22**

41 Final operational guidance **24**

1 Installation and project layout

AutoIA 0.21.0.dev1 requires Python 3.10 or newer and NumPy 1.24 or newer. SciPy is optional; it is needed only if the user's model itself imports SciPy and wants the supported SciPy normal/error-function syntax.

To install the built wheel into an environment:

```
python -m pip install autoia-0.21.0.dev1-py3-none-any.whl
```

To verify the installed version:

```
import autoia
print(autoia.__version__)
# 0.21.0.dev1
```

The most important public imports are:

```
from autoia import (
    autoia,
    autoia_subdivide,
    autoia_adaptive_subdivide,
    prepare,
    Interval,
    RigorousBinary64Backend,
    RangePolicy,
    SubdivisionPolicy,
    BranchPolicy,
)
```

2 First validated calculation

The smallest useful example is:

```
import numpy as np
from autoia import autoia, Interval


def model(p):
    x = p[0]
    return np.exp(x) + x*x


result = autoia(
    model,
    [Interval(-0.2, 0.3)],
    order=4,
)

print("lower =", result.lower)
print("upper =", result.upper)
print("width =", result.width)
print("machine rigorous =", result.machine_rigorous)
```

If `autoia(...)` returns normally, the reported interval is intended to contain the exact real program output for every input in the supplied interval. If the requested statement cannot be certified, the function raises a typed `AutoIAError` subclass and returns no rigorous result.

3 Specifying uncertain inputs

For one uncertain scalar parameter use

```
Interval(lower, upper)
```

with finite endpoints satisfying `lower <= upper`.

For m uncertain inputs, pass a sequence of m intervals:

```
box = [
    Interval(90.0, 110.0),
    Interval(900.0, 1100.0),
    Interval(9.5, 10.5),
]
result = autoia(model, box)
```

The uncertainty model is currently a Cartesian box. Correlated physical input sets such as ellipsoids, simplices encoded intrinsically, or arbitrary non-rectangular feasible sets are not yet first-class input-domain objects. A user may still write algebraic relations in the model, but the declared uncertain parameters themselves are supplied as independent interval coordinates.

4 Choosing a representation

The public `representation=` option accepts:

Value	Meaning and recommended use
`"polynomial"`	Default. Retains a sparse polynomial in the original uncertain inputs plus a rigorous interval remainder. Usually the best starting point for smooth engineering models.
`"generator"`	Retains the same physical-input polynomial plus shared auxiliary error generators. Use when downstream reuse/cancellation of approximated vector/intermediate quantities matters.
`"interval"`	Uses ordinary interval enclosures without retaining explicit polynomial dependence. Robust fallback and useful near some exact function-domain boundaries, but usually less sharp for repeated dependence.

Example:

```
result = autoia(
    model,
    [Interval(-1.0, 1.0)],
    order=3,
    representation="generator",
)
```

```
print(result.generator_ids)
print(result.generator_matrix)
```

The generator representation is not automatically superior in every model. It preserves additional error provenance at the cost of extra bookkeeping and can accumulate many generator IDs in long nonlinear computations.

5 Polynomial order

The `order` argument is a nonnegative integer controlling the maximum total degree retained in the polynomial representation.

```
result = autoia(model, box, order=4)
```

Higher order can reduce the interval remainder by retaining more physical-input dependence, but increases polynomial coefficient count and range-analysis cost. The appropriate value is model-dependent. Orders around 2–5 are the natural starting range for many small engineering examples in the current prototype.

Changing `order` affects efficiency and sharpness, not the mathematical inclusion contract: discarded polynomial terms are rigorously transferred to remainder bounds.

6 Supported scalar arithmetic

The core supported scalar operations include:

- addition and subtraction;
- multiplication;
- division when the denominator can be proved separated from zero;
- integer powers, including negative integer powers when zero is excluded;
- constant non-integer real powers on the supported positive-real branch.

Examples:

```
def model(p):
    x = p[0]
    a = x*x + 2.0*x - 1.0
    b = 1.0 / (2.0 + x*x)
    c = (1.5 + x)**1.25
    d = (2.0 + x)**-0.5
    return np.array([a, b, c, d])
```

An uncertain exponent such as `x ** p[1]` is currently unsupported.

7 Supported elementary and statistical functions

The ordinary NumPy and standard-library forms of these scalar functions are supported when visible through the model's module/global namespace:

```
np.sqrt(x)
np.exp(x)
np.log(x)
np.sin(x)
np.cos(x)

math.sqrt(x)
math.exp(x)
math.log(x)
math.sin(x)
math.cos(x)
math.erf(x)
```

AutoIA also exports:

```
from autoia import (
    erf,
    standard_normal_pdf,
    standard_normal_cdf,
    normal_pdf,
    normal_cdf,
    norm_pdf,
    norm_cdf,
)
```

For a normal distribution with constant mean `mu` and standard deviation `sigma > 0`:

```
def model(p):
    x = p[0]
    return np.array([
        normal_pdf(x, mean=10.0, std=2.0),
        normal_cdf(x, mean=10.0, std=2.0),
    ])
```

The current normal convenience APIs require constant distribution parameters. Uncertain mean/standard deviation parameters are not yet supported by those front-end helpers.

8 SciPy compatibility

If SciPy is installed and the user model imports it, these common forms are recognized:

```
import scipy

scipy.special.erf(x)
scipy.special.ndtr(x)

scipy.stats.norm.pdf(x)
scipy.stats.norm.cdf(x)
scipy.stats.norm.pdf(x, loc=mu, scale=sigma)
scipy.stats.norm.cdf(x, loc=mu, scale=sigma)
```

Frozen legacy normal distributions are supported:

```
from scipy.stats import norm
```

```
rv = norm(loc=2.0, scale=1.5)


def model(p):
    x = p[0]
    return np.array([rv.pdf(x), rv.cdf(x)])
```

When available in the installed SciPy version, the newer `scipy.stats.Normal` object API is also recognized for PDF/CDF use.

SciPy does not become proof code. On validated inputs the local dispatcher routes these calls to AutoIA's own rigorous error-function/normal kernels. Numeric-only calls continue to use ordinary SciPy.

9 NumPy arrays and ordinary engineering assembly

Inside AutoIA's private execution namespace, common array constructors default to `dtype=object` when the user did not explicitly request a dtype. This allows standard engineering assembly:

```
import numpy as np


def model(p):
    k1, k2, force = p

    A = np.zeros((2, 2))
    b = np.zeros(2)

    A[0, 0] = k1 + k2
    A[0, 1] = -k2
    A[1, 0] = -k2
    A[1, 1] = k2

    b[0] = force

    return np.linalg.solve(A, b)
```

Supported constructors include common forms of `array`, `asarray`, `zeros`, `ones`, `empty`, `full`, their `*_like` variants, `eye`, and `identity`.

Ordinary object-array indexing, slicing, reshape, transpose, stacking, concatenation, and item assignment are supported in the tested engineering cases.

10 Broadcasting, reductions, and matrix composition

Supported broadcasted scalar arithmetic includes cases where a numeric constant NumPy array created outside the model is combined with one validated scalar:

```
COEFF = np.array([1.5, -2.0, 0.25])


def model(p):
    return COEFF * p[0]
```

The supported reduction subset includes:

```
np.sum(A, axis=...)
np.prod(A, axis=...)
np.mean(A, axis=...)
np.trace(A)
```

with common `axis` and `keepdims` behavior. For validated reductions, omit `dtype` or use `dtype=object`. General `where=` and `out=` semantics are not frozen.

The tested 1D/2D composition subset includes:

```
np.dot(a, b)
np.matmul(A, x)
A @ x
np.inner(a, b)
np.outer(a, b)
```

11 Norms

The supported norm forms are:

```
np.linalg.norm(x)              # vector Euclidean norm
np.linalg.norm(x, ord=2)       # vector Euclidean norm
np.linalg.norm(A)              # matrix Frobenius norm
np.linalg.norm(A, ord="fro") # matrix Frobenius norm
```

For a vector $x = (x_1, \ldots, x_n)^T$, the supported Euclidean norm is

$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}.$$

Matrix spectral `ord=2`, general p-norms, nuclear norms, and max-based norms are not currently supported.

12 Semantic linear solves

Use ordinary NumPy syntax:

```
u = np.linalg.solve(A, b)
```

AutoIA recognizes this call and applies its semantic linear-system certificate. The entries of `A` and `b` may contain retained polynomial nonlinearity from earlier arithmetic or elementary functions.

Multiple right-hand sides are supported:

```
X = np.linalg.solve(A, B)
```

where `B` is a matrix with one right-hand side per column.

The solve may fail with `CertificationError` if AutoIA cannot prove the matrix family non-singular/contractive on the supplied box. This is not necessarily evidence that the true matrix is singular. Subdivision, a smaller input box, or a later stronger certifier may succeed.

13 Uncertain Python branches

Ordinary scalar conditionals are supported for the documented subset:

```
def model(p):
    x = p[0]

    if x < -0.25:
        y = -1.0 + x*x
    elif x <= 0.5:
        y = x*x
    else:
        y = 1.0 + x

    return y*y + 0.1*x
```

Supported scalar comparisons include `<`, `<=`, `>`, `>=`, `==`, and `!=`, plus scalar truth testing `if x:`. Python short-circuit `and`/`or` works through the same scalar truth mechanism in the tested cases.

If a predicate cannot be rigorously decided on the current box, AutoIA may split the input box or enumerate both feasible execution paths. A branching model returns an `AutoIAPathwiseResult` with branch metadata in addition to global lower/upper bounds.

14 Branch policy and resource limits

The default branch policy is equivalent to:

```
from autoia import BranchPolicy

policy = BranchPolicy(
    max_branches=64,
    max_path_decisions=24,
    max_split_depth_per_decision=2,
    max_total_splits=64,
    prefer_geometric_split=True,
)
```

Use it with:

```
result = autoia(model, box, branch_policy=policy)
```

If a hard path resource budget is exhausted before every feasible path is certified, AutoIA raises `ResourceLimitError`. It does not silently drop paths.

15 Range policies

The default final range policy is adaptive. The public policy object is:

```
from autoia import RangePolicy

policy = RangePolicy(
    internal_method="termwise",
    final_method="adaptive",
    bernstein_max_coefficients=256,
    proof_bernstein_max_coefficients=256,
```

```
)
```

The supported internal methods are:

- `"termwise"`;
- `"monotonicity"`;
- `"bernstein"`;
- `"portfolio"`.

The final method may additionally be `"adaptive"`.

Use either a `RangePolicy` object or the legacy convenience keyword options such as `range_method`; do not pass both styles in the same call.

16 Prescribed subdivision

To force a Cartesian subdivision, use:

```
from autoia import autoia_subdivide

piecewise = autoia_subdivide(
    model,
    [Interval(-1.0, 1.0), Interval(0.0, 2.0)],
    split_counts=[4, 2],
    order=4,
)
```

The `split_counts` sequence contains one positive integer per input coordinate. Each resulting subbox is independently certified.

17 Adaptive subdivision

Use:

```
from autoia import autoia_adaptive_subdivide, SubdivisionPolicy

policy = SubdivisionPolicy(
    max_branches=16,
    max_depth=6,
    max_probe_directions=2,
    min_structural_gain=0.05,
    min_relative_width_gain=1.0e-3,
    split_method="cheap",
)

result = autoia_adaptive_subdivide(
    model,
    box,
    order=4,
    subdivision_policy=policy,
)
```

The release-oriented default `split_method="cheap"` scores hypothetical splits from the existing polynomial and only certifies the selected child pair. Use `split_method="lookahead"` to reproduce the expensive historical one-step lookahead research oracle.

The current adaptive-subdivision controller does not generally compose with uncertain Python control flow. For path-dependent models use ordinary path-aware `autoia(...)` or prescribed `autoia_subdivide(...)` unless the current documentation explicitly states otherwise.

18 Understanding result objects

For an ordinary result:

```
print(result.lower)
print(result.upper)
print(result.width)
print(result.output_shape)
print(result.lower_shaped)
print(result.upper_shaped)
print(result.width_shaped)
print(result.machine_rigorous)
print(result.representation_names)
```

The flat `lower`/`upper` arrays are retained for compatibility. Use the shaped properties to recover the model's returned array shape.

In generator mode:

```
print(result.generator_ids)
print(result.generator_matrix)
```

Rows of `generator_matrix` correspond to flattened outputs and columns correspond to shared generator IDs.

The `objects` field contains the validated scalar representations. It is intended primarily for research/diagnostic use; ordinary users usually need only bounds and diagnostics.

19 Understanding pathwise results

A path-aware result additionally exposes information such as:

```
print(result.branch_results)
print(result.branch_boxes)
print(result.branch_decisions)
print(result.branch_traces)
print(result.path_events)
```

These records are useful when a wide global interval is caused by genuinely different program regimes. They should also be examined when path budgets become large or a branch-local singularity prevents certification.

20 Prepared sessions for repeated queries

For repeated certification of the same program/configuration, use `prepare` instead of calling one-shot `autoia` repeatedly:

```
from autoia import prepare, Interval

session = prepare(
    model,
    order=4,
    representation="generator",
    cache_max_entries=256,
)

r1 = session.certify([
    Interval(-1.0, 1.0),
    Interval(0.0, 2.0),
])

r2 = session.certify([
    Interval(-0.25, 0.25),
    Interval(0.5, 1.0),
])

print(session.stats)
print(r2.cache_info)
```

The session prepares the model's local dispatch once and reuses one audited trusted backend.

21 Certificate cache behavior

The session cache has three important query outcomes.

Fresh certificate.

```
result.cache_info["kind"] == "fresh-certificate"
```

The engineering model was executed and a new proof was produced.

Exact certificate hit.

```
result.cache_info["kind"] == "exact-certificate"
```

The exact same stored input endpoints were already certified; the model is not re-executed.

Restricted certificate hit.

```
result.cache_info["kind"] == "restricted-certificate"
```

A previously certified parent box contained the new query, and AutoIA rigorously re-centered/re-ranged the stored polynomial/generator certificate on the new interior box without executing the model.

The cache is bounded by `cache_max_entries`. Set it to zero to disable persistent cache storage:

```
session = prepare(model, cache_max_entries=0)
```

Use:

```
session.clear_cache()
```

to remove stored certificates.

22 Many-box finite collections

A finite collection may be supplied as nested endpoint arrays:

```
boxes = np.array([
    [[-1.0, -0.8], [0.0, 0.2]],
    [[-0.5, -0.3], [0.4, 0.6]],
    [[ 0.1,  0.2], [0.8, 1.0]],
], dtype=float)

results = session.certify_many(
    boxes,
    parallel="serial",
)
```

or as ordinary lists of `Interval` objects.

For bounds-only output:

```
lower, upper = session.certify_many_bounds(boxes)
print(lower.shape)
print(upper.shape)
```

If there are N boxes and r flattened scalar outputs, the arrays have shape (N, r).

23 CPU parallel many-box execution

The supported modes are:

```
parallel="serial"
parallel="thread"
parallel="process"
```

Example process mode:

```
results = session.certify_many(
    boxes,
    parallel="process",
    workers=8,
    chunk_size=64,
    use_cache=True,
    allow_restriction=True,
)
```

The model must be pickleable for process mode. The safest pattern is a module-level function:

```
# good for multiprocessing

def engineering_model(p):
    ...
```

A nested closure or lambda is generally not pickleable under the spawn execution mode. Use serial/thread mode or move the model to module scope.

Each worker process prepares the model once and creates its own audited trusted backend. Session-local caches are not automatically shared across processes. For heavily overlapping boxes, one serial/shared session with certificate reuse may outperform process mode despite using fewer cores; benchmark the actual workload.

24 Streaming millions of boxes

Do not construct a list of one million full result objects if the downstream workflow only needs bounds. Use a generator and the streaming API:

```
from autoia import Interval, prepare


def generate_boxes():
    for i in range(1_000_000):
        a = -1.0 + 2.0*i/1_000_000
        yield [Interval(a, a + 1.0e-3)]


session = prepare(model, order=3)

for lower, upper in session.iter_bounds_many(
    generate_boxes(),
    parallel="process",
    workers=8,
    chunk_size=256,
):
    # Write to disk/database or update an aggregate.
    consume(lower, upper)
```

The iterator keeps one executor alive and only a bounded number of chunks in flight. This controls orchestration memory, although each individual AutoIA certification still uses the current Python-object representation internally.

25 Session statistics

Inspect:

```
print(session.stats)
```

The returned `PreparedSessionStats` fields are:

- `model_preparations`;
- `certify_requests`;
- `engineering_model_executions`;
- `exact_cache_hits`;
- `restricted_cache_hits`;
- `cache_misses`;
- `cache_entries`.

These statistics are especially useful for deciding whether a repeated-query workload is benefiting from mathematical certificate reuse or merely from CPU parallelism.

26 Trusted reduction backend configuration

The default backend chooses the historical scalar trusted reduction for short reductions and the vectorized outward pairwise implementation for longer ones. You can create an explicit backend:

```
from autoia import RigorousBinary64Backend

backend = RigorousBinary64Backend(
    reduction_method="auto",
    vectorized_reduction_min_terms=16,
)

result = autoia(model, box, backend=backend)
```

For research comparisons:

```
reference_backend = RigorousBinary64Backend(
    reduction_method="reference"
)

fast_backend = RigorousBinary64Backend(
    reduction_method="vectorized"
)
```

Different rigorous reduction orders can produce slightly different certified endpoints. Bitwise identity is not a general requirement; containment is.

27 Typed errors and how to respond

Catch the common base class if an application wants one failure handler:

```
from autoia import AutoIAError

try:
    result = autoia(model, box)
except AutoIAError as exc:
    print(type(exc).__name__)
    print(exc)
```

Use the subtype to diagnose the problem:

Exception	Meaning / likely response
`InvalidInputError`	Malformed public call, invalid interval/order/policy/backend option. Correct the call.
`UnsupportedOperationError`	Program used syntax/operation outside the current supported subset. Rewrite using supported operations or implement a validated extension.
`CertificationError`	Supported mathematical operation was recognized but the requested proof failed, e.g. zero-crossing division or failed solve contraction. Subdivide or improve the certifier.
`ResourceLimitError`	Path/subdivision budget exhausted. Increase a justified budget or reduce the box.

Exception	Meaning / likely response
`NumericalLimitError`	Trusted finite computation overflow/underflow or environment numerical limitation prevented a proof.
`ModelExecutionError`	The user's ordinary Python model raised an exception or returned incompatible path shapes.
`InternalAutoIAError`	AutoIA detected an internal inconsistency; treat this as a software defect and preserve a reproducible case.

28 Current supported-language boundaries

Do not rely on the following as stable validated semantics unless later releases explicitly add them:

- `np.max`, `np.min`, `np.maximum`, `np.minimum`;
- general `abs`/sign/nonsmooth array operations;
- uncertain Boolean masks and `np.where` with uncertain masks;
- uncertain array indices or slices;
- arbitrary `einsum`/`tensordot` signatures;
- spectral/eigenvalue matrix norms;
- general uncertainty-dependent `while` loop termination;
- functions hidden inside compiled extensions or helpers whose globals are not visible to the local shadow dispatcher;
- arbitrary runtime-local imports inside the model body;
- general nonlinear root/ODE/eigen/optimization semantic primitives.

The correct behavior for unsupported semantics is fail closed, not silently coerce validated objects to ordinary floating point.

29 Known boundary issue for sqrt/log

A user interval may mathematically lie exactly on a valid domain boundary, such as $[0, 4]$ for $\sqrt{x}$. The polynomial representation uses a rigorous symmetric centered envelope that can extend by a tiny amount below the original lower endpoint because of outward rounding. AutoIA will not silently clip that negative proof interval.

If this occurs, practical options are:

- use the interval representation for that calculation if appropriate;
- subdivide or move the physical domain away from the exact boundary;
- wait for the planned richer boundary/domain representation before relying on polynomial-mode exact-boundary behavior.

30 Performance guidance for users

For one or a few boxes:

- start with `representation="polynomial"`;
- use the default backend/range policy;
- increase polynomial order only when remainder/range width justifies the extra cost;
- inspect branching/subdivision only when the global result is too wide or a proof fails.

For many boxes over one program:

- use `prepare`;
- keep overlapping/interior queries in one session so certificate restriction can occur;
- inspect `session.stats`;
- use process mode for many unrelated fresh boxes on a multicore CPU;
- use streaming bounds APIs for very large collections;
- benchmark cache reuse versus process parallelism on the actual workload.

The current implementation is not yet a true vectorized million-box polynomial kernel. Step 9c freezes the API and safe reuse semantics before the numeric batch-buffer/GPU refactor.

31 Developer setup and regression commands

From the project root, run the complete suite:

```
python -m unittest discover -s tests -p 'test*.py'
```

For the final 0.21.0.dev1 checkpoint the expected gate is:

```
Ran 223 tests
OK
```

Run the repeated-query audit:

```
python tools/audit_manybox.py --boxes 40 --workers 4
```

Build a wheel without network build isolation when the environment already has setuptools:

```
python -m pip wheel . --no-deps --no-build-isolation -w dist
```

Do not accept a performance change based only on a microbenchmark. Run the historical regression suite plus the relevant independent exact/high-precision audit and an end-to-end engineering benchmark.

32 Source tree for developers

Location	What to change there
`autoia/api.py`	Public one-box/subdivision signatures, validation, exception translation.
`autoia/session.py`	Prepared session, cache, repeated-query orchestration, CPU parallelism.
`autoia/trusted/`	Only proof-critical binary64 primitives and audited reductions/transcendentals. Keep dependencies minimal.
`autoia/polynomial/`	Sparse polynomial algebra, ranges, Bernstein, recentering.
`autoia/enclosures/`	Representation semantics and arithmetic.
`autoia/semantic/linear_solve.py`	Mathematical linear-system semantic primitive and its residual/contraction proof.
`autoia/semantic/dispatch.py`	Private NumPy/math syntax routing.
`autoia/semantic/scipy_dispatch.py`	Public SciPy syntax compatibility.
`autoia/controller/branching.py`	Uncertain Python paths and piecewise execution.
`autoia/controller/subdivision.py`	Prescribed/adaptive subdivision execution.
`autoia/controller/split_scoring.py`	Cheap non-proof split-direction prediction.
`autoia/elementary/taylor.py`	Taylor coefficients/remainders for smooth functions.
`tests/`	Regression, boundary, exact-rational, high-precision, and integration tests.
`tools/`	Larger randomized/audit/performance scripts that are not shipped as proof dependencies.

33 Adding a new supported elementary function

A recommended development sequence is:

1. State the real mathematical function and domain.
2. Implement a trusted scalar/interval kernel or a trusted coefficient/domain bound in `autoia/trusted/elementary.py`.
3. Add independent high-precision point and interval tests.
4. Add Taylor coefficient intervals and a rigorous tail bound in `autoia/elementary/taylor.py`.
5. Connect the function to representation-level composition in `autoia/enclosures/elementary.py`.
6. Add NumPy/math/SciPy syntax only if it corresponds to a real public API users commonly write.
7. Add polynomial, interval, generator, object-array, and composition tests.

8. Add a standalone audit if the function is mathematically delicate.

Do not make a platform library's ordinary transcendental return value a trusted endpoint merely because it is accurate in common cases.

34 Adding a new semantic primitive

For a future operation such as a nonlinear root, eigensystem, or ODE solver:

1. Define the exact mathematical object being certified.
2. Decide whether the goal is to certify the program algorithm as written or replace a recognized library call by its mathematical semantics.
3. Build an ordinary floating-point proposal if useful.
4. Derive an independent proof identity, e.g. residual/fixed-point/Krawczyk/ energy argument.
5. Implement the proof in `autoia/semantic/` using trusted low-level bounds.
6. Add local dispatch for the recognized library syntax.
7. Preserve a generic validated fallback when reasonable; otherwise fail with `UnsupportedOperationError` or `CertificationError` as appropriate.
8. Add adversarial and near-singular tests, not only nominal examples.

The key rule is: *semantic recognition may change how the operation is computed, but not what mathematical claim is proved.*

35 Adding or changing a representation

Every new scalar representation must answer four questions:

1. What exact set $\Gamma_E(q)$ does the object represent?
2. How is every supported arithmetic operation guaranteed to map input sets to an output superset?
3. How are conversions to/from existing representations inclusion-preserving?
4. How is a final scalar interval range obtained rigorously?

Only after these are answered should the representation receive a promotion priority in `ValidatedValue` dispatch.

For the shared-generator family, never merge two generator IDs solely because their coefficients or radii are numerically equal. Equal IDs mean a proved shared auxiliary variable; different IDs mean independent auxiliary coordinates.

36 Changing trusted arithmetic

Treat `autoia/trusted/` as a small numerical proof library.

Before changing it:

- identify the exact-real quantity the function must contain;
- preserve or extend runtime environment audits;
- add exact-rational tests for algebraic operations;
- add independent high-precision tests for transcendental operations;
- test overflow, subnormal, zero, sign, and endpoint-adversarial cases;
- keep the historical scalar oracle available when introducing a faster vectorized path.

Do not call higher AutoIA layers from the trusted package. Architecture boundary tests are designed to catch such inversions.

37 Changing the many-box cache or parallel backend

The current cache uses a bounded LRU dictionary plus a linear scan for the smallest containing certificate. A future spatial index can replace this without changing the public `PreparedAutoIA` API.

However, any new cache must preserve these rules:

1. Cache entries are scoped to one fixed prepared model/configuration.
2. Exact endpoint equality may reuse a certificate directly.
3. Containment-based reuse must verify the new *represented centered symmetric domain*, not merely the raw user interval, lies in the parent certified domain.
4. Polynomial recentering must capture coordinate-shift and coefficient rounding rigorously.
5. Per-request metadata should not mutate result objects previously handed to users.
6. A cache miss must fall back to a normal fresh proof, not to an approximate answer.

For process parallelism, remember that every worker owns a separate cache. A future shared spatial certificate store will need explicit synchronization and serialization rules; it should not be introduced implicitly through global mutable proof state.

38 GPU development guidance

The current user-facing prepared/batch API is intentionally compatible with a future GPU backend, but the current `dtype=object` representation is not. A serious GPU implementation should introduce numeric structure-of-arrays buffers behind the existing tracing objects.

A sensible staged plan is:

1. GPU proposal kernels first: batched nominal solves, predictor coefficients, preconditioner proposals, controller estimates. Revalidate on the CPU trusted backend.

2. Numeric CPU batch buffers next, so the same memory model can be transferred to the GPU.

3. Only later implement a proof-producing GPU backend using explicitly audited directed-rounding primitives. Do not assume ordinary CuPy/cuBLAS reductions are rigorous.

GPU support should plug into `PreparedAutoIA.certify_many` or a compatible backend selection interface rather than introduce an unrelated user API.

39 Release checklist for a code change

Before treating a change as a checkpoint:

1. Run the complete unit/regression suite.

2. Run the relevant exact-rational or high-precision audit.

3. Run at least one end-to-end engineering smoke test.

4. If performance is claimed, compare against the previous checkpoint on the same workload and report both kernel and end-to-end timing.

5. Build the wheel from the final source tree.

6. Install that wheel into a clean target and run a smoke test through the installed package rather than the source checkout.

7. Inspect wheel contents to ensure tests/development scaffolding are not shipped unintentionally.

8. Update the mathematical contract, code map, user manual, and roadmap when the public semantics change.

9. Keep a patch archive from the previous checkpoint as well as a complete source archive.

40 Complete copyable example: nonlinear solve, branch, statistics, and repeated queries

The following example uses several current capabilities together.

```
import numpy as np
from scipy.stats import norm

from autoia import Interval, prepare


def engineering_model(p):
    x, y = p

    A = np.zeros((2, 2))
    A[0, 0] = 2.0 + 0.10*x*x
    A[0, 1] = 0.20
    A[1, 0] = -0.10*y
    A[1, 1] = 1.50 + 0.05*np.exp(x)
```

```
    b = np.zeros(2)
    b[0] = 1.0 + 0.10*y
    b[1] = 0.5 + 0.05*np.sin(x)

    u = np.linalg.solve(A, b)

    if x >= 0.0:
        response = u[0] + u[1]*u[1]
    else:
        response = u[0] - u[1]

    probability = norm.cdf(response, loc=0.5, scale=0.2)
    return np.array([response, probability])


session = prepare(
    engineering_model,
    order=4,
    representation="generator",
    cache_max_entries=256,
)

master = session.certify([
    Interval(-0.5, 0.5),
    Interval(-0.2, 0.2),
])

print("master lower:", master.lower)
print("master upper:", master.upper)

queries = [
    [[-0.45, -0.30], [-0.10, 0.00]],
    [[-0.20, -0.05], [ 0.00, 0.10]],
    [[ 0.05,  0.15], [-0.05, 0.05]],
    [[ 0.30,  0.40], [ 0.05, 0.15]],
]

lower, upper = session.certify_many_bounds(
    queries,
    parallel="serial",
)

print("lower bounds:")
print(lower)
print("upper bounds:")
print(upper)
print("session stats:")
print(session.stats)
```

For a module-level model and many unrelated boxes, change `parallel="serial"` to `parallel="process"` and specify a worker count. If the boxes lie inside a cached parent and are eligible for certificate restriction, a shared serial session can instead avoid the model/solve execution entirely for those queries.

41 Final operational guidance

AutoIA is most reliable when the user treats a returned certificate and a failed certificate differently. A returned result is a mathematical claim about the supported program semantics on the declared box. A failure is not automatically a statement that the engineering model is invalid; it may mean that the current representation, domain information, controller budget, or semantic certificate is insufficiently sharp.

For development, preserve the same distinction in code: proposal logic may be optimized aggressively, but every accepted result must remain traceable to the trusted enclosure and semantic proof contracts. The prepared-session layer adds a second important rule: reuse is safe only when a previously proved certificate is applied to a domain that has itself been rigorously shown to lie inside the parent certified domain.